

Panduan Git

untuk Tim Empat Orang

Cabang mengatur WAKTU.
CODEOWNERS mengatur WILAYAH.

Dari nol sampai bisa bekerja tanpa konflik.

Untuk	Isi singkat
Empat pengembang fullstack, masing-masing satu modul	Istilah git, isi cabang main dan dev, alur kerja harian, cara menghindari dan menyelesaikan konflik
Work owner	Pembagian modul, proteksi cabang GitHub, pengisian CODEOWNERS

Repositori github.com/adesaragih/NusantaraRe · Versi 1 · 1 Oktober 2026

Sumber dan versi yang selalu mutakhir: [docs/bersama/PANDUAN-CABANG-GIT.md](#) di repositori.

Daftar Isi

Bab	Judul	Untuk siapa
1	Cara memakai panduan ini	semua
2	Kamus istilah git — baca ini dulu	pemula
3	Salah paham terbesar: cabang bukan folder	semua
4	Peta cabang NusantaraRe	semua
5	Isi setiap cabang	semua
6	Siapa memegang modul apa	semua
7	Alur kerja harian — langkah demi langkah	pengembang
8	Berkas bersama — daftar merah	pengembang
9	Kalau terjadi konflik — pertolongan pertama	pengembang
10	Menulis pesan commit	pengembang
11	Proteksi cabang di GitHub	work owner
12	Kartu contekan	semua

1 Cara memakai panduan ini

Repositori NusantaraRe sekarang dikerjakan empat orang sekaligus. Sampai kemarin ia dipegang satu orang, jadi aturan git belum pernah dibutuhkan. Mulai sekarang dibutuhkan — tanpa aturan, empat orang yang menulis ke repositori yang sama akan saling menimpa pekerjaan.

Kabar baiknya: struktur aplikasi ini sudah dirancang supaya konflik hampir mustahil terjadi. Setiap modul punya foldernya sendiri, nomor migrasinya sendiri, dan slot menunya sendiri. Yang tersisa hanyalah mengikuti aturan cabang di panduan ini.

Kalau Anda belum pernah memakai git sama sekali

Baca bab 2 sampai paham, lalu bab 3. Dua bab itu yang paling menentukan. Sisanya bisa dibuka saat dibutuhkan.

Kalau Anda sudah terbiasa dengan git

Langsung ke bab 4 (peta cabang), bab 6 (pembagian modul), dan bab 8 (daftar berkas yang tidak boleh disentuh). Bab 3 tetap layak dibaca sekilas — salah paham di sana juga sering menimpa yang sudah berpengalaman.

CATATAN Panduan ini kembaran dari berkas di repositori

Isi yang sama ada di docs/bersama/PANDUAN-CABANG-GIT.md. Bila keduanya berbeda, yang di repositori yang berlaku — ia ikut diperbarui bersama kode.

2 Kamus istilah git — baca ini dulu

Git punya kosakata sendiri. Tanpa ini, seluruh panduan terasa seperti bahasa asing. Kolom tengah sengaja memakai perumpamaan sehari-hari, bukan definisi teknis.

Istilah	Perumpamaannya	Dalam proyek ini
repository	Lemari arsip berisi seluruh berkas proyek beserta seluruh riwayat perubahannya.	Satu lemari untuk semuanya: github.com/adesaragih/NusantaraRe
commit	Satu foto dari seluruh isi lemari pada satu saat, diberi keterangan singkat kenapa difoto.	Satu commit = satu pekerjaan selesai, misalnya satu tiket
branch (cabang)	Satu jalur foto yang berjalan sendiri. Bukan sebagian lemari — seluruh lemari, tapi versinya sendiri.	main, dev, dan satu cabang per modul
checkout / switch	Berpindah melihat lemari pada jalur yang lain. Isi folder di laptop Anda berubah mengikutinya.	git switch modul/nbtreatyin
merge	Menggabungkan dua jalur jadi satu, supaya pekerjaan di jalur lain ikut masuk.	Cabang modul digabung ke dev
conflict (konflik)	Dua orang mengubah baris yang sama di berkas yang sama. Git tidak bisa menebak mana yang benar, jadi ia berhenti dan bertanya.	Hampir tidak terjadi di sini — kecuali di berkas bersama (bab 8)
pull	Mengambil foto terbaru dari lemari bersama ke laptop Anda.	Dilakukan setiap pagi
push	Mengirim foto Anda ke lemari bersama supaya dilihat orang lain.	Setelah commit, sebelum membuka pull request
pull request (PR)	Permintaan resmi: “tolong periksa pekerjaan saya, lalu gabungkan.” Orang lain membaca dan menyetujui.	Satu-satunya jalan masuk ke dev dan main
CODEOWNERS	Daftar penjaga: berkas di folder ini, yang berhak memeriksa adalah si A.	Berkas .github/CODEOWNERS, satu pemilik per modul
remote / origin	Lemari bersama di GitHub. “origin” nama panggilannya.	origin/dev artinya cabang dev di GitHub

CATATAN Satu kalimat yang merangkum semuanya

Cabang menentukan KAPAN pekerjaan naik ke rilis. CODEOWNERS menentukan SIAPA boleh mengubah apa. Dua hal berbeda, dan keduanya diperlukan.

3 Salah paham terbesar: cabang bukan folder

Hampir semua orang yang baru masuk tim bertanya hal yang sama:

“Cabang main isinya folder apa saja? Apakah seluruh folder, atau hanya intinya? Lalu dev isinya apa?”

Pertanyaan itu berangkat dari bayangan yang keliru: seolah cabang membagi folder — main kebagian inti, dev kebagian modul. Bukan begitu.

JANGAN Yang SALAH dibayangkan

main berisi folder inti saja; dev berisi folder modul saja; tiap cabang membawa potongan yang berbeda dari proyek.

LAKUKAN Yang BENAR

Setiap cabang memuat SELURUH proyek — semua dua puluh modul, inti, docs, semuanya. Yang membedakan cabang bukan folder mana, melainkan versi kapan.

Perumpamaannya

Bayangkan sebuah buku. main dan dev adalah dua naskah LENGKAP dari buku yang sama — bukan “bab 1–5 di satu map, bab 6–10 di map lain”. dev adalah naskah draf terbaru yang masih disunting. main adalah naskah yang sudah dicetak dan dibagikan. Keduanya punya seluruh bab.

Buktinya di repositori kita, 1 Oktober 2026

	main	dev
Jumlah berkas	2.129	2.137
Folder tingkat pertama	.github, APP_RNM, dstin, discovery, docs, jefri, keluaran-docx, scripts	sama persis
Isi APP_RNM/modul/	dua puluh modul	dua puluh modul
Isi APP_RNM/inti/	ada	ada
Selisihnya	tertinggal 17 commit	lebih baru 17 commit

Daftar folder tingkat pertamanya identik. Selisih delapan berkas itu bukan folder yang ditinggalkan — itu berkas baru yang lahir di dev dan belum naik ke main.

Tujuh belas commit yang membedakannya menyentuh semua lapisan

Bukan satu folder tertentu — ada yang menyentuh inti, ada yang menyentuh modul, ada yang menyentuh dokumen:

```
inti: 901 - M_NAV_MENU datar, satu baris per modul, PARENT_ID dibuang
inti: GET /api/menu satu tingkat di bawah GROUPMENU
inti: sidebar satu tombol per modul, dikelompokkan GROUPMENU
docs: brief implementasi Master Contract Retro Life
...
```

Akibatnya untuk pekerjaan sehari-hari

Ketika pemegang modul nbtreatyin berpindah ke cabangnya, folder di laptopnya tetap berisi semua modul dan seluruh inti:

```
git switch modul/nbtreatyin

D:\NUSANTARA RE APP\NusantaraRe\
APP_RNM\modul\nbtreatyin\    <- hanya INI yang boleh dia ubah
APP_RNM\modul\claimlife\    <- tetap ada, tapi bukan miliknya
APP_RNM\modul\...           <- 18 modul lain, tetap ada
APP_RNM\inti\               <- tetap ada, wajib ada
docs\ discovery\ scripts\   <- tetap ada
```

Memang harus begitu. Kode modul nbtreatyin meng-import inti/backend/kontrak dan inti/frontend/components. Kalau cabang benar-benar hanya membawa sebagian folder, aplikasinya tidak akan bisa dijalankan sama sekali.

PERHATIAN Lalu apa yang membatasi dia?

Bukan cabang, melainkan berkas .github/CODEOWNERS. Di sana tertulis siapa pemilik tiap folder. Kalau dia mengubah folder milik orang lain, pull request-nya otomatis meminta persetujuan pemilik folder itu — dan akan ditolak.

4 Peta cabang NusantaraRe

Enam jenis cabang. Semuanya memuat seluruh proyek; yang berbeda perannya.

```
main                                rilis - yang dipasang ke server
|                                  terkunci, hanya menerima PR dari dev
|
+-- dev                            integrasi - pekerjaan yang sudah ditinjau
|                                  terkunci, hanya menerima PR
|
+-- modul/nbtreatyin              cabang tetap milik Dev B
|    +-- modul/nbtreatyin/tiket-01 umur 1-3 hari
|    +-- modul/nbtreatyin/tiket-02
|
+-- modul/claimprop              cabang tetap milik Dev D
|    +-- modul/claimprop/tiket-01
|
+-- inti/<topik>                  ubah berkas bersama - PR kecil

perbaikan/<topik>                 hotfix, dicabang dari main
                                digabung ke main DAN ke dev
```

Arah penggabungan — satu jalur, tidak pernah berbalik

```
modul/<nama>/<topik>  -> modul/<nama>  -> dev  -> main
inti/<topik>          -> dev
perbaikan/<topik>     -> main dan dev
```

JANGAN Cabang modul tidak pernah langsung ke main

Selalu lewat dev. dev yang memastikan pekerjaan empat orang bisa hidup berdampingan sebelum dinyatakan rilis.

Tabel ringkas

Nama cabang	Dicabang dari	Digabung ke	Umur
main	—	—	selamanya
dev	main	main	selamanya
modul/<nama>	dev	dev	sepanjang migrasi modul
modul/<nama>/<topik>	modul/<nama>	modul/<nama>	1–3 hari
inti/<topik>	dev	dev	1 hari
perbaikan/<topik>	main	main + dev	sependek mungkin

5 Isi setiap cabang

Cabang	Isinya	Siapa boleh menggabung ke sini
main	Seluruh proyek, pada titik terakhir yang ujinya hijau dan sudah disetujui. Patokannya: kalau hari ini ada demo ke pengguna atau pemasangan ke server, yang diambil dari sini.	Hanya pull request dari dev. Tidak pernah di-push langsung.
dev	Seluruh proyek, versi lebih baru — pekerjaan yang sudah selesai dan ditinjau, tapi belum dinyatakan rilis. Inilah cabang yang ditarik tim setiap pagi.	Pull request dari cabang modul dan dari cabang inti.
modul/<nama>	Seluruh proyek. Yang berubah hanya folder APP_RNM/modul/<nama>/ milik pemiliknya.	Pemilik modul itu saja.
inti/<topik>	Seluruh proyek, dengan perubahan kecil pada berkas bersama — pustaka baru, kontrak lintas modul, konfigurasi.	Siapa pun, tapi wajib ditinjau tim inti.
perbaikan/<topik>	Seluruh proyek, dengan perbaikan mendesak atas sesuatu yang rusak di server.	Setelah digabung ke main, WAJIB digabung juga ke dev.

CATATAN Kenapa cabang modul dibagi dua tingkat

Satu modul butuh belasan tiket — claimlife menghabiskan dua belas giliran. Kalau semuanya ditumpuk di satu cabang, pull request-nya jadi raksasa dan tidak mungkin ditinjau dengan teliti. Cabang topik menjaga tiap pull request tetap kecil.

PERHATIAN Jangan lupa menggabungkan hotfix ke dev

Kalau perbaikan hanya masuk ke main, pada rilis berikutnya dev akan menyimpannya kembali dengan kode yang rusak. Ini kesalahan paling sering terjadi di pola main+dev.

6 Siapa memegang modul apa

Dasar pembagiannya satu kalimat: modul yang berbagi tabel atau kontrak harus dipegang satu orang. ADR 0041 menyatakan endorsemen berbagi tabel dengan new business, dan setiap modul komiteclaim memakai kontrak modul claim pasangannya. Memisahkan pasangan itu ke dua orang sama dengan menjadwalkan konflik.

Orang	Lingkup	Modul yang dipegang	Rentang migrasi
Dev A	Fakultatif	nbfacin rnwfacin endorsmentfacin claimfacin komiteclaimfacin	180–299 520–559 640–679
Dev B	Treaty In	nbtreatyin edmtreatyin treatyin treatyinadjustment	320–479
Dev C	Master & Life	mastercontractretrolife masterproductnamelife endorsementlife merawat: premiumlistlife merawat: treatycontractout	100–179 480–519
Dev D	Klaim Prop & Non-Prop	claimprop komiteclaimprop claimnonprop komiteclaimnonprop merawat: claimlife merawat: komiteclaimlife	560–639 680–759

LAKUKAN Kenapa rentang migrasi penting

Rentang keempat orang tidak bersinggungan sama sekali. Itulah yang membuat folder backend/migrations/ tidak pernah bentrok — Dev B menulis berkas bernomor 320-479, Dev D menulis 560-759, tidak ada nomor yang sama.

Dev A memegang lima modul, tapi tiga di antaranya berbagi tabel yang sama, jadi bebannya setara Dev B.

PERHATIAN Nomor migrasi dan slot menu sudah ditetapkan

Nomornya tertulis di berkas MODUL.md tiap modul, dan ditegakkan otomatis oleh penjaga di inti/backend/penjaga. Jangan menulis migrasi di luar rentang Anda — uji akan merah. Mengubah rentang adalah pull request ke tim inti.

7 Alur kerja harian — langkah demi langkah

Sekali saja, saat pertama kali mulai

1. Ambil salinan repositori ke laptop:

```
git clone https://github.com/adesaragih/NusantaraRe.git
cd NusantaraRe
```

2. Buat cabang tetap untuk modul Anda, dicabang dari dev:

```
git switch dev
git pull
git switch -c modul/nbtreatyin
git push -u origin modul/nbtreatyin
```

Setiap pagi — kebiasaan yang paling menurunkan konflik

```
git switch modul/nbtreatyin
git fetch origin
git merge origin/dev      # ambil pekerjaan tiga orang lain
```

PERHATIAN Pakai merge, bukan rebase, untuk cabang yang sudah di-push

Rebase menulis ulang riwayat. Kalau riwayat itu sudah dilihat orang lain, mereka terpaksa memperbaiki salinannya. Untuk cabang topik yang BELUM pernah di-push, rebase justru lebih rapi.

Setiap tiket

1. Buat cabang topik dari cabang modul Anda:

```
git switch modul/nbtreatyin
git switch -c modul/nbtreatyin/tiket-01
```

2. Kerjakan — **hanya di dalam folder modul Anda**.

3. Simpan pekerjaan, sebutkan foldernya supaya tidak ada berkas lain ikut terbawa:

```
git add -A -- APP_RNM/modul/nbtreatyin
git commit -m "nbtreatyin: tiket 01 - tabel penyimpanan new business"
```

4. Kirim ke GitHub dan buka pull request ke cabang modul Anda:

```
git push -u origin modul/nbtreatyin/tiket-01
```

5. Setelah disetujui dan digabung, hapus cabang topiknya:

```
git switch modul/nbtreatyin
git pull
git branch -d modul/nbtreatyin/tiket-01
```

Setiap beberapa tiket — naikan ke dev

Buka pull request dari modul/nbtreatyin ke dev. Jangan menunggu seluruh modul selesai.

PERHATIAN Pull request kecil dan sering

Cabang modul yang hidup tiga minggu tanpa digabung akan bentrok hebat di go.mod, inti/backend/daftar/daftar.go, dan APP_RNM/frontend/daftar.ts. Sekali per tiket, bukan sekali per modul.

Bonus: mengerjakan dua modul tanpa harus menyimpan pekerjaan setengah jadi

Perintah worktree membuat folder kerja kedua dari repositori yang sama. Folder .worktrees/ sudah diabaikan git, jadi aman:

```
git worktree add .worktrees/nbtreatyin modul/nbtreatyin
```

8 Berkas bersama — daftar merah

Pemisahan satu folder per modul sudah menyelesaikan hampir seluruh masalah konflik. Yang tersisa hanya berkas yang dipakai semua modul. Inilah daftarnya.

JANGAN Jangan pernah menyentuh berkas ini di cabang modul

```
go.mod go.sum package.json package-lock.json vite.config.ts tsconfig.json
Makefile
APP_RNM/inti/** (kecuali daftar/modul_<nama>_gen.go milik Anda)
APP_RNM/frontend/** APP_RNM/cmd/** APP_RNM/uji/**
.github/** docs/bersama/**
```

Lalu bagaimana kalau memang perlu mengubahnya?

Pisahkan jadi pull request sendiri yang kecil:

```
git switch dev
git pull
git switch -c inti/tambah-pustaka-decimal
# ubah go.mod saja
git commit -am "inti: tambah pustaka decimal untuk presisi uang"
git push -u origin inti/tambah-pustaka-decimal
# buka PR ke dev -> digabung lebih dulu -> semua orang 'git pull'
```

Menumpangkan perubahan berkas bersama di pull request modul membuat empat orang menunggu satu tinjauan — dan itu persis yang ingin kita hindari.

Dua jebakan khusus

Jebakan	Kenapa berbahaya	Cara menghindari
package-lock.json	Puluhan ribu baris dan dibangkitkan mesin — konfliknya mustahil diselesaikan dengan tangan. Hanya ada SATU yang berlaku: APP_RNM/package-lock.json. Yang muncul di akar repositori adalah hasil npm install di folder salah.	Jangan pernah di-merge manual (bab 9). Jalankan npm install dari dalam APP_RNM, atau dengan --prefix APP_RNM.
Nomor ADR	Empat orang menulis ADR berikutnya serentak menghasilkan empat berkas 0044-*.md dan empat suntingan pada adr/00-INDEKS.md.	Cadangkan blok nomor per orang, atau ambil nomornya lewat pull request satu baris ke dev sebelum menulis isinya.

9 Kalau terjadi konflik — pertolongan pertama

Konflik bukan kerusakan. Git hanya sedang berkata: “dua orang mengubah baris yang sama, saya tidak berani menebak.” Jangan panik, dan jangan menghapus folder lalu clone ulang.

Langkah umum

1. Lihat berkas mana yang bentrok:

```
git status
```

2. Buka berkasnya. Git menandai bagian yang bentrok seperti ini:

```
<<<<<< HEAD
      versi milik Anda
=====
      versi dari dev
>>>>>> origin/dev
```

3. Pilih yang benar, hapus ketiga baris penanda (<<<<<<, =====, >>>>>>), simpan.
4. Tandai selesai lalu lanjutkan:

```
git add <nama-berkas>
git commit
```

Kasus khusus: package-lock.json

Berkas ini jangan pernah disunting dengan tangan. Buang versi yang bentrok, lalu biarkan npm membangunnya ulang:

```
git checkout --theirs APP_RNM/package-lock.json
npm install --prefix APP_RNM
git add APP_RNM/package-lock.json
```

Kalau benar-benar buntu

```
git merge --abort      # batalkan, kembali ke keadaan sebelum merge
```

LAKUKAN Kapan harus bertanya, bukan menebak

Kalau yang bentrok adalah berkas milik orang lain — modul orang lain, atau berkas di daftar merah bab 8 — jangan diputuskan sendiri. Tanya pemiliknya. Menebak di sana berarti menghapus pekerjaan orang tanpa dia tahu.

10 Menulis pesan commit

Riwayat adalah satu-satunya cara menelusuri “perubahan ini kenapa” setelah pembuatnya lupa — atau pindah kerja. Selama repositori dipegang sendiri, pesan asal-asalan tidak terasa merugikan. Begitu timnya berempat, pesan adalah alat kerja.

Formatnya

```
<lingkup>: <apa yang berubah>
```

Lingkup diisi nama modul, atau kata inti, atau kata docs.

Baik	Buruk	Kenapa
inti: sidebar satu tombol per modul,	KOMIT	Tidak menyebut apa pun. Enam bulan lagi tak seorang pun tahu isinya.
nbtreatyin: tiket 01 - tabel	HEHE	Sama sekali tidak bisa dicari.
docs: ralat PK/FK dari katalog DEV	update	Semua commit adalah update. Tidak membedakan apa pun.
claimprop: perbaiki hitung OS klaim	fix bug	Bug yang mana, di modul mana?

PERHATIAN Riwayat dev saat ini memuat empat pesan yang tidak layak

KOMIT, HEHE, dan dua kali LALALLALLA — peninggalan masa repositori ini dipegang satu orang. Tidak dihapus karena menulis ulang riwayat yang sudah dibagikan justru merepotkan semua orang. Tapi mulai sekarang pesan seperti itu tidak lagi diterima dalam pull request.

11 Proteksi cabang di GitHub

Bab ini untuk work owner. Seluruh aturan di panduan ini hanya imbauan sampai GitHub menegakkannya. Nyalakan di Settings → Branches → Add branch protection rule.

Pengaturan	main	dev
Push langsung	ditolak	ditolak
Pull request wajib	ya	ya
Jumlah persetujuan	1, dari orang lain	1, dari orang lain
Tinjauan CODEOWNERS wajib	ya	ya
Uji wajib hijau (go test, vitest)	ya	ya
Riwayat linear	ya	—
Hapus cabang otomatis sesudah merge	ya	ya

PERHATIAN Satu kotak centang yang mudah terlewat

“Require approval from someone other than the last pusher” harus menyala. ADR 0040 melarang menyetujui pekerjaan sendiri, dan tanpa kotak ini aturan itu tidak tertegakkan.

Yang harus dilakukan sebelum tim mulai

1. Isi akun GitHub sebenarnya di `.github/CODEOWNERS` — ada dua puluh penanda `@PEMILIK-*` dan satu `@TIM-INTI`. Selama masih penanda, tinjauan CODEOWNERS tidak menegakkan apa pun.
2. Tetapkan blok nomor ADR per orang, supaya empat orang tidak merebut nomor 0044 bersamaan.
3. Nyalakan proteksi cabang pada `main` dan `dev` menurut tabel di atas.
4. Bagikan panduan ini, dan tunjuk satu orang sebagai rujukan pertama kalau ada yang bingung dengan git.

12 Kartu contekan

Cetak halaman ini dan tempel di dekat meja.

Perintah yang dipakai setiap hari

Mau apa	Perintahnya
Mulai hari, ambil pekerjaan orang lain	<pre>git switch modul/<nama> git fetch origin git merge origin/dev</pre>
Mulai tiket baru	<pre>git switch -c modul/<nama>/tiket-NN</pre>
Lihat saya sedang di cabang mana	<pre>git branch --show-current</pre>
Lihat apa saja yang sudah saya ubah	<pre>git status</pre>
Simpan pekerjaan	<pre>git add -A -- APP_RNM/modul/<nama> git commit -m "<nama>: ..."</pre>
Kirim ke GitHub	<pre>git push -u origin <nama-cabang></pre>
Lihat riwayat singkat	<pre>git log --oneline -10</pre>
Batalkan merge yang kacau	<pre>git merge --abort</pre>
Buang perubahan satu berkas yang belum di-commit	<pre>git restore <berkas></pre>

Tiga aturan yang kalau dilanggar pasti menimbulkan konflik

	Aturan
1	Hanya ubah folder modul Anda sendiri. Berkas bersama (bab 8) lewat cabang inti/<topik> dengan pull request terpisah.
2	Tarik dev setiap pagi. <code>git fetch origin</code> lalu <code>git merge origin/dev</code> . Satu kebiasaan ini paling banyak mencegah konflik.
3	Pull request kecil dan sering. Sekali per tiket, bukan sekali per modul. Cabang yang hidup tiga minggu pasti bentrok.

Dan satu kalimat yang merangkum seluruh panduan ini:

Cabang mengatur WAKTU — kapan pekerjaan naik ke rilis.

CODEOWNERS mengatur WILAYAH — siapa boleh mengubah apa.